Manual Técnico - ExtreamFS

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Ingeniería en Ciencias y Sistemas
Manejo e Implementación de Archivos - 2S2026

Estudiante: Cesar Armando García Franco
Carnet: 202110378
Proyecto: Proyecto 1 - ExtreamFS
Fecha: Septiembre 2026

Índice

1. Introducción
2. Arquitectura del Sistema
3. Estructuras de Datos
4. Comandos Implementados
5. Sistema de Reportes
6. Diagrama de Flujo
7. Conclusiones

---

1. Introducción

ExtreamFS es una aplicación web que simula un sistema de archivos **EXT2** mediante archivos binarios `.mia` . El backend está desarrollado en **C++** y expone una API RESTful. El frontend es una aplicación web en **React** que consume dicha API.

1.1 Objetivos

- Comprender el funcionamiento interno de un sistema de archivos EXT2.
- Implementar operaciones de bajo nivel sobre archivos binarios.
- Desarrollar una API en C++ que exponga las funcionalidades.
- Crear una interfaz web para interactuar con el sistema.

1.2 Tecnologías Utilizadas

Componente	Tecnología
Backend	C++ 17
Servidor HTTP	cpp-httplib
JSON	nlohmann/json
Reportes	Graphviz
Frontend	React + Vite
Sistema Operativo	Linux

---

2. Arquitectura del Sistema

2.1 Visión General

```
graph TD
    A[Usuario] --> B[Frontend React]
    B -->|POST /comando| C[Backend C++]
    C --> D[Lexer]
    D --> E[Parser]
    E --> F[Ejecutor]
    F --> G[Archivo .mia]
    G --> H[Respuesta JSON]
    H --> B
```

2.2 Módulos del Backend

Módulo	Descripción
Lexer	Convierte texto en tokens
Parser	Valida parámetros del comando
Ejecutores	Coordina la ejecución
Commands	Lógica de cada comando
Disk	Gestión de MBR y particiones
EXT2	Gestión de SuperBlock, Inodos, Bloques

Módulo	Descripción
Server	Servidor HTTP

2.3 Flujo de Comunicación

1. El usuario ingresa un comando en el frontend.
2. El frontend envía el comando al backend vía `POST /comando`.
3. El backend procesa el comando (Lexer → Parser → Ejecutor).
4. El backend devuelve la respuesta en JSON.
5. El frontend muestra el resultado al usuario.

---

3. Estructuras de Datos

3.1 MBR (Master Boot Record)

Se escribe en el byte 0 del disco.

Campo	Tipo	Descripción
mbr_size	int	Tamaño del disco en bytes
mbr_date	time_t	Fecha de creación
mbr_disk_signature	int	Número único del disco
fit	char	Tipo de ajuste: B, F, W
mbr_partitions[4]	Partition[4]	4 particiones

3.2 Partition

Campo	Tipo	Descripción
part_status	char	0=inactiva, 1=montada
part_type	char	P=primaria, E=extendida
part_fit	char	B, F, W
part_start	int	Byte de inicio
part_size	int	Tamaño en bytes
part_name[16]	char[16]	Nombre

Campo	Tipo	Descripción
part_correlative	int	Número correlativo
part_id[4]	char[4]	ID al montar

3.3 EBR (Extended Boot Record)

Se usa para particiones lógicas dentro de una extendida.

Campo	Tipo	Descripción
part_mount	char	0=no montada, 1=montada
part_fit	char	B, F, W
part_start	int	Byte de inicio
part_size	int	Tamaño
part_next	int	Byte del siguiente EBR
part_name[16]	char[16]	Nombre

3.4 SuperBlock

Se escribe una vez al inicio de la partición formateada.

Campo	Tipo	Descripción
s_filesystem_type	int	2 = EXT2
s_inodes_count	int	Total de inodos
s_blocks_count	int	Total de bloques
s_free_inodes_count	int	Inodos libres
s_free_blocks_count	int	Bloques libres
s_mtime	time_t	Último montaje
s_umtime	time_t	Último desmontaje
s_mnt_count	int	Veces montado
s_magic	int	0xEF53
s_inode_size	int	Tamaño del inodo
s_block_size	int	Tamaño del bloque

Campo	Tipo	Descripción
s_first_ino	int	Primer inodo libre
s_first_blo	int	Primer bloque libre
s_bm_inode_start	int	Inicio bitmap inodos
s_bm_block_start	int	Inicio bitmap bloques
s_inode_start	int	Inicio tabla inodos
s_block_start	int	Inicio tabla bloques

3.5 Inodo

Campo	Tipo	Descripción
i_uid	int	UID del propietario
i_gid	int	GID del grupo
i_size	int	Tamaño en bytes
i_atime	time_t	Último acceso
i_ctime	time_t	Fecha de creación
i_mtime	time_t	Última modificación
i_block[15]	int[15]	12 directos + 3 indirectos
i_type	char	0=carpeta, 1=archivo
i_perm[3]	char[3]	Permisos UGO

3.6 Bloque

Existen 3 tipos:

Bloque de Carpeta:

Campo	Tipo	Descripción
b_content[4]	Content[4]	4 entradas

Content:

Campo	Tipo	Descripción
b_name[12]	char[12]	Nombre

Campo	Tipo	Descripción
b_inode	int	Apuntador al inodo

Bloque de Archivo:

Campo	Tipo	Descripción
b_content[64]	char[64]	Contenido

Bloque de Apuntadores:

Campo	Tipo	Descripción
b_pointer[16]	int[16]	Apuntadores

3.7 Cálculo de Inodos y Bloques

La fórmula para calcular el número de inodos y bloques es:

$$
\text{tamaño_particion} = \text{sizeof(SuperBlock)} + n + 3n + n \cdot \text{sizeof(Inode)} + 3n \cdot \text{sizeof(Block)}
$$

Donde:

- n = número de inodos
- $3n$ = número de bloques
- $\text{sizeof(SuperBlock)} \approx 100$ bytes
- $\text{sizeof(Inode)} \approx 100$ bytes
- $\text{sizeof(Block)} = 64$ bytes

Despejando n:

$$
n = \left\lfloor \frac{\text{tamaño_particion} - \text{sizeof(SuperBlock)}}{1 + 3 + \text{sizeof(Inode)} + 3 \cdot 64} \right\rfloor
$$

---

4. Comandos Implementados

4.1 MKDISK

Crea un archivo `.mia`.

Parámetro	Obligatorio	Descripción
-size	Sí	Tamaño
-unit	No	K o M
-fit	No	BF, FF, WF
-path	Sí	Ruta

```
mkdisk -path=/home/usuario/Disco1.mia -size=10 -unit=M -fit=BF
```

4.2 RMDISK

Elimina un archivo `.mia`.

Parámetro	Obligatorio	Descripción
-path	Sí	Ruta

```
rmdisk -path=/home/usuario/Disco1.mia
```

4.3 FDISK

Gestiona particiones.

Parámetro	Obligatorio	Descripción
-size	Sí	Tamaño
-unit	No	B, K, M
-path	Sí	Ruta del disco
-type	No	P, E, L
-fit	No	BF, FF, WF
-name	Sí	Nombre

```
fdisk -path=/home/usuario/Disco1.mia -name=Part1 -size=5 -unit=M -type=P
```

4.4 MOUNT

Monta una partición en RAM.

Parámetro	Obligatorio	Descripción
-path	Sí	Ruta del disco
-name	Sí	Nombre de la partición

```
mount -path=/home/usuario/Disco1.mia -name=Part1
```

Genera un ID con el formato: `[últimos 2 dígitos carnet][número][letra]`.

4.5 MOUNTED

Lista las particiones montadas.

```
mounted
```

4.6 MKFS

Formatea una partición como EXT2.

Parámetro	Obligatorio	Descripción
-id	Sí	ID
-type	No	full o fast

```
mkfs -id=781A -type=full
```

4.7 LOGIN

Inicia sesión.

Parámetro	Obligatorio	Descripción
-user	Sí	Usuario
-pass	Sí	Contraseña

Parámetro	Obligatorio	Descripción
-id	Sí	ID de partición

```
login -user=root -pass=123 -id=781A
```

4.8 LOGOUT

Cierra sesión.

```
logout
```

4.9 MKGRP

Crea un grupo (solo root).

Parámetro	Obligatorio	Descripción
-name	Sí	Nombre del grupo

```
mkgrp -name=usuarios
```

4.10 RMGRP

Elimina un grupo (solo root).

Parámetro	Obligatorio	Descripción
-name	Sí	Nombre del grupo

```
rmgrp -name=usuarios
```

4.11 MKUSR

Crea un usuario (solo root).

Parámetro	Obligatorio	Descripción
-user	Sí	Usuario

Parámetro	Obligatorio	Descripción
-pass	Sí	Contraseña
-grp	Sí	Grupo

```
mkusr -user=user1 -pass=pass123 -grp=usuarios
```

4.12 RMUSR

Elimina un usuario (solo root).

Parámetro	Obligatorio	Descripción
-user	Sí	Usuario

```
rmusr -user=user1
```

4.13 CHGRP

Cambia el grupo de un usuario (solo root).

Parámetro	Obligatorio	Descripción
-user	Sí	Usuario
-grp	Sí	Nuevo grupo

```
chgrp -user=user1 -grp=admin
```

4.14 MKFILE

Crea un archivo.

Parámetro	Obligatorio	Descripción
-path	Sí	Ruta del archivo
-r	No	Crear carpetas padre
-size	No	Tamaño en bytes
-cont	No	Ruta de archivo externo

```
mkfile -path=/home/user/archivo.txt -size=100
```

4.15 MKDIR

Crea una carpeta.

Parámetro	Obligatorio	Descripción
-path	Sí	Ruta de la carpeta
-p	No	Crear carpetas padre

```
mkdir -path=/home/user -p
```

4.16 CAT

Muestra el contenido de archivos.

Parámetro	Obligatorio	Descripción
-file1..N	Sí	Rutas de archivos

```
cat -file1=/users.txt
cat -file1=/users.txt -file2=/home/test.txt
```

4.17 REP

Genera reportes.

Parámetro	Obligatorio	Descripción
-name	Sí	Tipo de reporte
-path	Sí	Ruta de salida
-id	Sí	ID de partición
-path_file_ls	No	Ruta para file/ls

Tipos de reporte:

Nombre	Descripción

Nombre	Descripción
mbr	MBR
disk	Disco
sb	SuperBloque
bm_inode	Bitmap de inodos
bm_block	Bitmap de bloques
inode	Inodos
block	Bloques
tree	Árbol de directorios
file	Contenido de archivo
ls	Listado de archivos

```
rep -id=781A -path=/home/reportes/tree.jpg -name=tree
rep -id=781A -path=/home/reportes/file.txt -name=file -path_file_ls=/user
```

---

5. Sistema de Reportes

Los reportes se generan con **Graphviz** en formato DOT y se convierten a imágenes con el comando `dot`.

Reporte	Descripción
Tree	Árbol completo de directorios
Inode	Tabla de inodos ocupados
Block	Tabla de bloques ocupados
Bitmaps	Estado de inodos y bloques
MBR	Estructura del MBR
Disk	Estructura del disco
SB	SuperBloque
File	Contenido de un archivo
Ls	Listado de archivos

---

6. Diagrama de Flujo

```
flowchart TD
    A[Usuario] --> B[Frontend React]
    B -->|POST /comando| C[Backend C++]
    C --> D[Lexer]
    D --> E[Parser]
    E --> F[Ejecutor]
    F --> G[Archivo .mia]
    G --> H[Respuesta JSON]
    H --> B
```

---

7. Conclusiones

- Se implementó un sistema de archivos EXT2 completo en C++.
- Se comprendió el funcionamiento de estructuras de bajo nivel (MBR, Inodos, Bloques).
- Se desarrolló una API RESTful con cpp-httplib.
- Se generaron reportes visuales con Graphviz.
- Se construyó una interfaz web interactiva con React.
